STUDENT NAME: **YASH CHAUHAN**

STUDENT ROLL NUMBER: **28**

SECTION: CSE **C-1**

STUDENT REG NUMBER: **230905194**

WEEK NO: **4**

LAB4 -

Token.h -->

```
#ifndef TOKEN_H
#define TOKEN_H
#include <stdio.h>
#include <ctype.h>
#include <string.h>
struct token {
char token_name[50];
      int index;
      unsigned int
      row, col; char
      type[30];
};
char *keywords[] = {
"int","float","char","double","void",
"if","else","while","for","return"
};
int keywordCount = 10;

int isKeyword(char *);
int isArithmetic(char *);
int isRelational(char *);
int isLogical(char *);
int isSpecialSymbol(char);
int isIdentifier(char *);

int isKeyword(char *lex) {
      for (int i = 0; i < keywordCount; i++)
            if (strcmp(lex, keywords[i]) == 0)
```

```
                return 1;
     return 0;
}

int isArithmetic(char *lex) {
     return (!strcmp(lex,"+") || !strcmp(lex,"-") ||
     !strcmp(lex,"*") || !strcmp(lex,"/") ||
     !strcmp(lex,"%"));
}

int isRelational(char *lex) {
     return (!strcmp(lex,"<") || !strcmp(lex,">") ||
     !strcmp(lex,"<=") || !strcmp(lex,">=") ||
     !strcmp(lex,"==") || !strcmp(lex,"!="));
}

int isLogical(char *lex) {
     return (!strcmp(lex,"&&") || !strcmp(lex,"||") ||
     !strcmp(lex,"!"));
}
int isSpecialSymbol(char ch) {
     return (ch=='('||ch==')'||ch=='{'||ch=='}'||
     ch==';'||ch==',' );
}

int isIdentifier(char *lex) {
     if (!isalpha(lex[0]) && lex[0] != '_')
          return 0;
     for (int i = 1; lex[i]; i++)
          if (!isalnum(lex[i]) && lex[i] != '_')
               return 0;
     return 1;
}

#endif
```

main.c -->

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "token.h"
```

```
#define TABLE_SIZE 10

FILE *fp;

int expectingIdentifier = 0;
char lastType[20] = "";

int isArray = 0;
int arraySize = 1;

typedef struct Symbol {
    char lexemeName[50];
    char type[20];
    int size;
    char returnType[20];
    struct Symbol *next;
} Symbol;

Symbol *hashTable[TABLE_SIZE];

int Hash(char *lexeme) {
    int sum = 0;
    for (int i = 0; lexeme[i]; i++)
        sum += lexeme[i];
    return sum % TABLE_SIZE;
}

Symbol *Search(char *lexeme) {
    int index = Hash(lexeme);
    Symbol *temp = hashTable[index];

    while (temp) {
        if (!strcmp(temp->lexemeName, lexeme))
            return temp;
        temp = temp->next;
    }
    return NULL;
}

void Insert(char *lexeme, char *type, int size, char *returnType) {
    if (Search(lexeme))
        return;

    int index = Hash(lexeme);
    Symbol *n = (Symbol *)malloc(sizeof(Symbol));

    strcpy(n->lexemeName, lexeme);
    strcpy(n->type, type);
    n->size = size;
    strcpy(n->returnType, returnType);
```

```
    n->next = hashTable[index];
    hashTable[index] = n;
}

void DisplaySymbolTable() {
    printf("\n%-15s %-10s %-5s %-12s\n",
           "LexemeName", "Type", "Size", "ReturnType");
    printf("-----------------------------------------------\n");

    for (int i = 0; i < TABLE_SIZE; i++) {
        Symbol *t = hashTable[i];
        while (t) {
            printf("%-15s %-10s %-5d %-12s\n",
                   t->lexemeName, t->type,
                   t->size, t->returnType);
            t = t->next;
        }
    }
}

int isDataType(char *str) {
    char *types[] = {
        "int", "char", "float", "double",
        "short", "long", "void"
    };
    int n = sizeof(types) / sizeof(types[0]);

    for (int i = 0; i < n; i++)
        if (!strcmp(str, types[i]))
            return 1;

    return 0;
}

int getTypeSize(char *type) {
    if (!strcmp(type, "char")) return 1;
    if (!strcmp(type, "short")) return 2;
    if (!strcmp(type, "int")) return 4;
    if (!strcmp(type, "float")) return 4;
    if (!strcmp(type, "double")) return 8;
    if (!strcmp(type, "long")) return 8;
    return 4;
}

int isLibraryFunction(char *name) {
    return (!strcmp(name, "printf") ||
            !strcmp(name, "scanf"));
}

int getNextToken(struct token *t) {
```

```
    char c;
    int i = 0;

    while ((c = fgetc(fp)) != EOF) {

        if (isspace(c)) continue;

        if (c == '"') {
            while ((c = fgetc(fp)) != '"' && c != EOF);
            continue;
        }

        if (isalpha(c) || c == '_') {
            t->token_name[i++] = c;
            while (isalnum(c = fgetc(fp)) || c == '_')
                t->token_name[i++] = c;

            t->token_name[i] = '\0';
            ungetc(c, fp);

            strcpy(t->type,
                   isKeyword(t->token_name) ? "KEYWORD" :
"IDENTIFIER");
            return 1;
        }

        if (isdigit(c)) {
            t->token_name[i++] = c;
            while (isdigit(c = fgetc(fp)))
                t->token_name[i++] = c;

            t->token_name[i] = '\0';
            ungetc(c, fp);
            strcpy(t->type, "NUM");
            return 1;
        }

        t->token_name[0] = c;
        t->token_name[1] = '\0';
        strcpy(t->type, "SPECIAL");
        return 1;
    }
    return 0;
}

int main() {
    struct token t;
    fp = fopen("input.c", "r");

    if (!fp) {
        printf("Cannot open file\n");
```

```
    return 0;
}

for (int i = 0; i < TABLE_SIZE; i++)
    hashTable[i] = NULL;

while (getNextToken(&t)) {

    if (!strcmp(t.type, "KEYWORD") && isDataType(t.token_name)) {
        strcpy(lastType, t.token_name);
        expectingIdentifier = 1;
        continue;
    }
    if (!strcmp(t.token_name, ",")) {
        expectingIdentifier = 1;
    }
    if (expectingIdentifier && !strcmp(t.type, "IDENTIFIER")) {

        struct token lookahead;
        long pos = ftell(fp);
        getNextToken(&lookahead);

        if (!strcmp(lookahead.token_name, "[")) {
            struct token num;
            getNextToken(&num);
            arraySize = atoi(num.token_name);
            isArray = 1;
            getNextToken(&lookahead);
        }

        if (!strcmp(lookahead.token_name, "(")) {
            if (isLibraryFunction(t.token_name))
                Insert(t.token_name, "func", 0, "NA");
            else
                Insert(t.token_name, "func", 0, lastType);
        }
        else {
            int baseSize = getTypeSize(lastType);
            int finalSize = baseSize * (isArray ? arraySize : 1);
            Insert(t.token_name, lastType, finalSize, "NA");
        }

        isArray = 0;
        arraySize = 1;
        continue;
    }

    if (!strcmp(t.token_name, ";"))
        expectingIdentifier = 0;
}
```

```
    DisplaySymbolTable();
    fclose(fp);
    return 0;
}
```

```
yashc@YashChauhan MINGW64 ~/Downloads/CDLAB
$ cat input.c
int a, b;
char c;

void fun() {
    int x, y;
    x = a + b;
}

int NotFun(){
    char arr[10];
    printf("dasfdaf");
    int z;
    z=10;
    return 0;
}

int main(){
    printf("Hi");
    double a1,b1;
    return 0;
}
yashc@YashChauhan MINGW64 ~/Downloads/CDLAB
$ gcc lab3.c -o lab3

yashc@YashChauhan MINGW64 ~/Downloads/CDLAB
$ ./lab3

LexemeName      Type       Size  ReturnType
---------------------------------------------
x               int        4     NA
main            func       0     int
y               int        4     NA
z               int        4     NA
NotFun          func       0     int
arr             char       10    NA
a1              double     8     NA
b1              double     8     NA
a               int        4     NA
b               int        4     NA
printf          func       0     NA
fun             func       0     void
c               char       1     NA
```